TALLER PRÁCTICO DE REPASO Y CONSOLIDACIÓN EN C++

Informe de la Actividad 6 del Programa Integrador Anual (P.I.A.)

E.E.S.T. N.º 1 “Eduardo Ader” — Vicente López

Datos del trabajo

Estudiante: Malek Cheheid
Curso: 5° año, 3° “A”
Materia: Laboratorio de Programación (LPR)
Profesor: Yamil Ganduglia
Actividad: N.º 6 — Taller de repaso e integración
Fecha: 15 de octubre de 2026

Repositorio del proyecto

https://github.com/malekcheheid-art/repasogeneral.git

1. Introducción

En este trabajo realicé los tres retos del taller de repaso: la suma recursiva, la búsqueda secuencial en un arreglo y el intercambio de valores con punteros. El objetivo de la actividad es comprender cómo se comporta la memoria a bajo nivel al ejecutar código en C++.

Durante el desarrollo noté que una función recursiva necesita un caso base claro; si no lo tiene, la pila (stack) crece sin control hasta producir un desbordamiento. También comprobé que un arreglo guarda sus elementos en celdas contiguas y que un puntero permite modificar una variable original fuera de su ámbito local.

El programa final está en src/main.cpp. Compila sin errores con g++ y lo ejecuté en Windows PowerShell para observar la salida real, incluyendo las direcciones hexadecimales de la RAM.

El código, las capturas y este informe se mantienen en el repositorio de GitHub indicado en la portada.

2. Metodología de aprendizaje 5R

• R1 — Registrar: compilé y ejecuté el programa para ver la salida real y las direcciones hexadecimales.

• R2 — Reducir: identifiqué el caso base, el break y la variable temporal como el núcleo de cada reto.

• R3 — Responder: justifiqué por qué el programa actúa así a bajo nivel (ver sección 5).

• R4 — Reflexionar: relacioné la lógica con el manejo de buffers y datos de sensores del Proyecto Integrador Anual.

• R5 — Revisar: verifiqué la compilación, realicé el push a GitHub y GitLab, y actualicé docs/CHANGELOG.md.

3. Desarrollo de los retos

3.1 Reto 1 — Suma recursiva de 1 hasta n

La función se invoca a sí misma disminuyendo n en cada paso. El caso base (n <= 0) corta la cadena de llamadas y previene el Stack Overflow.

```
int sumaRecursiva(int n) {
    if (n <= 0) return 0;   // caso base obligatorio
    return n + sumaRecursiva(n - 1);
}
```

Entrada utilizada: n = 5. Salida obtenida: 1 + 2 + 3 + 4 + 5 = 15.

3.2 Reto 2 — Búsqueda secuencial en memoria contigua

Recorrí un arreglo de 10 enteros celda por celda. Al encontrar el valor 18 imprimo su índice y su dirección en RAM. Luego corto el ciclo con break para evitar comparaciones innecesarias y optimizar el tiempo de ejecución.

```
int vectorDatos[10] = {12, 45, 78, 90, 122, 5, 18, 33, 67, 99};
int numeroBuscar = 18;
// for (int i = 0; i < TAM; i++) {
//   if (vectorDatos[i] == numeroBuscar) { ... break; }
// }
```

Salida obtenida: valor 18 hallado en el índice [6], con su dirección hexadecimal correspondiente.

3.3 Reto 3 — Intercambio de valores mediante punteros

Pasé las direcciones de x e y a la función. Con la desreferenciación (*) leí y escribí directamente sobre las celdas originales, intercambiando físicamente los valores en RAM.

```
void intercambiarValores(int* ptrA, int* ptrB) {
    int temporal = *ptrA;
    *ptrA = *ptrB;
    *ptrB = temporal;
}
```

Entrada: x = 100, y = 500. Salida: x = 500, y = 100.

4. Evidencias de compilación y ejecución

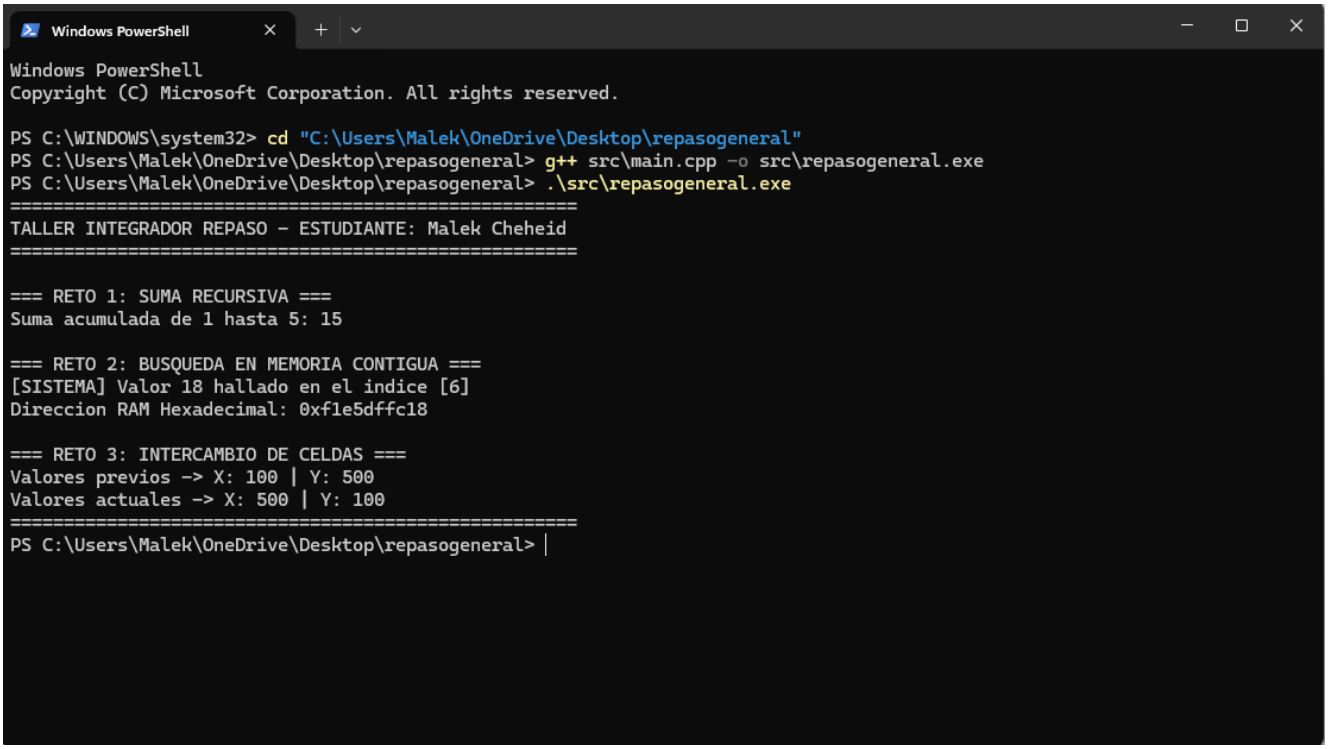
Compilé el programa con g++ (TDM-GCC 10.3.0) y lo ejecuté en Windows PowerShell. La captura muestra la salida completa de los tres retos.

Comandos de compilación y ejecución utilizados:

```
g++ src/main.cpp -o src/repasogeneral.exe
.\src\repasogeneral.exe
```

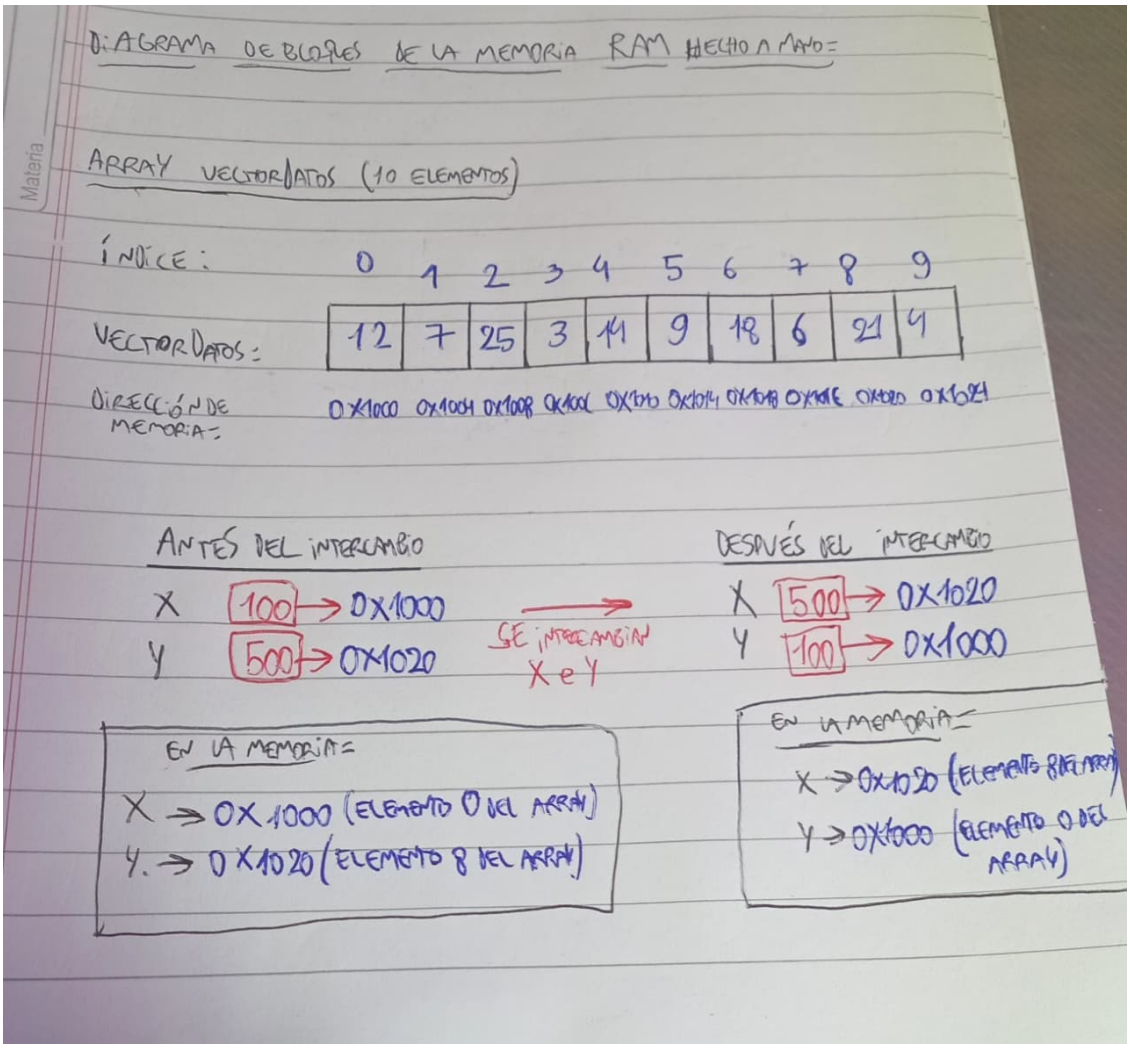
Si el equipo no permite compilar localmente, la actividad permite usar OnlineGDB y guardar las capturas como evidencia.

Figura 1. Ejecución local del programa (salida de los tres retos).



El diagrama de memoria muestra el bloque contiguo del arreglo, la celda hallada con su dirección y el intercambio de x e y.

Figura 2. Diagrama de bloques de la memoria RAM (hecho a mano).



4.1 Verificación de la configuración dual-remote

Además de las capturas, comprobé la configuración de control de versiones ejecutando git remote -v. El resultado obtenido fue el siguiente:

```
origin  https://github.com/malekcheheid-art/repasogeneral.git (fetch)
origin  https://github.com/malekcheheid-art/repasogeneral.git (push)
origin  https://gitlab.com/malekcheheid-art/repasogeneral.git (push)
```

Este resultado confirma un origen de fetch (GitHub) y dos destinos de push (GitHub y GitLab).

5. Cuestionario técnico de control

Las respuestas a continuación las redacté en mis propias palabras, justificando el comportamiento del programa a bajo nivel.

1. Caso Base: ¿Qué error crítico ocurre en la memoria RAM si una función recursiva carece de un caso base de corte?

Si falta el caso base, la función se llama a sí misma indefinidamente. Cada llamada apila un marco en el stack con parámetros, variables locales y la dirección de retorno. Como el stack tiene tamaño finito, la memoria se agota y se produce un Stack Overflow, que normalmente termina el programa de forma abrupta.

2. Eficiencia: ¿Por qué la instrucción break optimiza el rendimiento del microprocesador al realizar una búsqueda en arreglos?

El break corta el ciclo apenas se encuentra el elemento, evitando comparaciones innecesarias con el resto de las celdas. Esto reduce instrucciones ejecutadas y accesos a memoria, mejorando el

tiempo promedio aunque el peor caso siga siendo O(n).

3. Direccionamiento: ¿Cómo comprobar mediante el operador & que los elementos de un array ocupan celdas contiguas en la RAM?

Imprimiendo &vectorDatos[i] para cada índice se observa que las direcciones avanzan de forma regular (por lo general 4 bytes por int en x86/x64). Esa regularidad demuestra que el arreglo se almacena en posiciones contiguas, tal como define un bloque de memoria contigua.

4. Variables Temporales: ¿Qué función cumple la variable temporal en el algoritmo de intercambio de valores con punteros?

La variable temporal guarda el valor original de *ptrA antes de que sea sobreescrito. Sin ella ese dato se perdería y no se podría completar la rotación. Gracias a ella, x e y intercambian física y realmente sus valores en RAM.

5. Autoevaluación: ¿Cuál de los 3 retos te resultó más complejo de comprender a bajo nivel y de qué manera lograste resolverlo?

El reto de punteros me resultó el más complejo por la diferencia entre una variable y su dirección. Lo resolví dibujando bloques y flechas en papel (ptrA, ptrB, temporal) y verificando cada paso compilando y mostrando las direcciones por pantalla.

6. Tabla de Control de Revisiones (DocVer)

Tabla 1. *Control de Revisiones (DocVer)*

```
Versión  | Fecha      | Estado            | Descripción
v1.0.0   | 2026-10-15 | Baseline aprobado  | Estructura inicial del repo, 3 retos
unificados,
         |            |                   | capturas de ejecución y de memoria RAM,
informe APA v7.
```

7. Conclusión

Con este taller afiancé el manejo de la pila, la memoria contigua de los arreglos y el pasaje por dirección mediante punteros. Estas bases me resultan útiles para tareas futuras del Proyecto Integrador Anual, como la gestión de buffers y datos de sensores en tiempo real.

El repositorio quedó con la estructura solicitada y sincronizado en GitHub y GitLab mediante la configuración dual-remote.

CODIGO COMPLETO

```
/*================================================================
Materia: Laboratorio de Programacion (LPR) - 5to Anio, 3ro "A"
Institucion: E.E.S.T. N.1 "Eduardo Ader" - Vicente Lopez
Archivo: src/main.cpp
Actividad 6: Taller de Repaso e Integracion

Autor: Malek Cheheid
Profesor: Yamil Ganduglia
================================================================*/

#include <iostream>
using namespace std;

// --- Prototipos de funciones modulares ---
int sumaRecursiva(int n);
void intercambiarValores(int* ptrA, int* ptrB);

int main() {
    cout << "=====================================================" << endl;
    cout << "TALLER INTEGRADOR REPASO - ESTUDIANTE: Malek Cheheid" << endl;
    cout << "=====================================================" << endl;
```

```
    // --- RETO 1: RECURSIVIDAD ---
    cout << "\n=== RETO 1: SUMA RECURSIVA ===" << endl;
    int num = 5;
    cout << "Suma acumulada de 1 hasta " << num << ": " << sumaRecursiva(num) << endl;

    // --- RETO 2: ARRAYS Y BUSQUEDA SECUENCIAL ---
    cout << "\n=== RETO 2: BUSQUEDA EN MEMORIA CONTIGUA ===" << endl;
    const int TAM = 10;
    int vectorDatos[TAM] = {12, 45, 78, 90, 122, 5, 18, 33, 67, 99};
    int numeroBuscar = 18;
    bool hallado = false;

    for (int i = 0; i < TAM; i++) {
        if (vectorDatos[i] == numeroBuscar) {
            cout << "[SISTEMA] Valor " << numeroBuscar
                 << " hallado en el indice [" << i << "]" << endl;
            cout << "Direccion RAM Hexadecimal: " << &vectorDatos[i] << endl;
            hallado = true;
            break; // Interrupcion optimizada del ciclo O(n)
        }
    }

    if (!hallado) {
        cout << "[SISTEMA] El valor " << numeroBuscar
             << " no existe en el arreglo." << endl;
    }

    // --- RETO 3: INTERCAMBIO CON PUNTEROS ---
    cout << "\n=== RETO 3: INTERCAMBIO DE CELDAS ===" << endl;
    int x = 100, y = 500;
    cout << "Valores previos -> X: " << x << " | Y: " << y << endl;

    // Invocacion pasando direcciones de memoria
    intercambiarValores(&x, &y);

    cout << "Valores actuales -> X: " << x << " | Y: " << y << endl;

    cout << "=====================================================" << endl;
    return 0; // Codigo 0: Ejecucion exitosa
}

// Desarrollo de la funcion recursiva con caso base garantizado
int sumaRecursiva(int n) {
    if (n <= 0) return 0;   // Caso base obligatorio para prevenir Stack Overflow
    return n + sumaRecursiva(n - 1); // Llamada recursiva
}

// Desarrollo del intercambio fisico mediante desreferenciacion
void intercambiarValores(int* ptrA, int* ptrB) {
    int temporal = *ptrA;
    *ptrA = *ptrB;
    *ptrB = temporal;
}
```